

The background of the slide is white, framed by a thick dark blue border. Scattered throughout the slide are various squares in shades of blue and teal, some solid and some semi-transparent, creating a modern, geometric aesthetic.

Agustin Nicolas Carsaniga

TRABAJO PRÁCTICO – CONTROL DE VERSIONES CON GIT Y GITHUB

Analista en Sistemas - 3er Año - Ingenieria de
Software II

21/5/2026

<https://github.com/ElPelucaDLAP/practica-git>

Parte 1: Iniciación a Git y configuración básica

Consigna 1: Configurar Git en tu equipo

Consigna 2: Crear un repositorio local y primer commit

Parte 2: Trabajo con ramas

Consigna 3: Crear y trabajar en una nueva rama

Consigna 4: Fusionar ramas

Parte 3: Repositorio remoto y sincronización

Parte 4: Colaboración y trabajo en equipo

Consigna 5: Agregar un colaborador

Consigna 6: Clonar un repositorio de un colaborador

Parte 5: Conflictos y resolución

Consigna 7: Generando y resolviendo problemas de merge

Link al repositorio:

Parte 1: Iniciación a Git y configuración básica

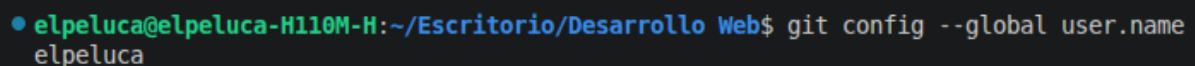
Consigna 1: Configurar Git en tu equipo

Ejecuta los siguientes comandos en Git Bash y pegá una captura:

```
git config --global user.name "Tu Nombre"
```

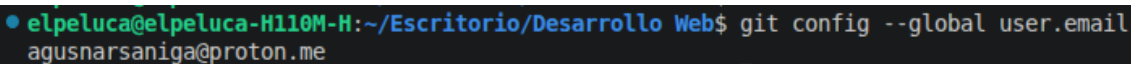
```
git config --global user.email "tuemail@example.com"
```

```
git config --list
```



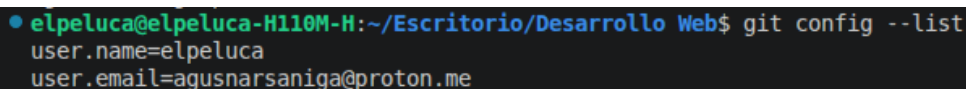
```
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web$ git config --global user.name elpeluca
```

Al ya tener configurado el nombre de usuario, utilizó el comando para saber con que nombre me encuentro asignado en el Git local.



```
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web$ git config --global user.email agusnarsaniga@proton.me
```

Al ya tener configurado el mail de usuario, utilizó el comando para saber qué mail está asignado en el git local.



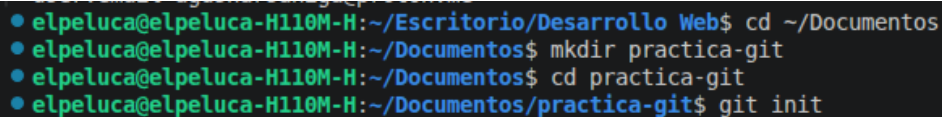
```
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web$ git config --list
user.name=elpeluca
user.email=agusnarsaniga@proton.me
```

Al listar toda la configuración me mostraran los parámetros previamente dichos.

Consigna 2: Crear un repositorio local y primer commit

Ejecuta el paso a paso y pegá las capturas correspondientes:

`git init` - inicializar repositorio:



```
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web$ cd ~/Documentos
• elpeluca@elpeluca-H110M-H:~/Documentos$ mkdir practica-git
• elpeluca@elpeluca-H110M-H:~/Documentos$ cd practica-git
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git init
```

Me sitúo con `cd` arriba de la carpeta donde desee inicializar el repositorio(que en este caso se crea por línea de comandos), y utilizo `git init` que indica que la carpeta en la que estoy parado se volverá un repositorio local de git.

Pregunta:

¿Dónde queda guardado el control de versiones? ¿En la nube o en tu equipo?

En este caso el control de versiones fue iniciado de forma local, por lo cual queda guardado en el equipo.

`git status` (antes del `add`):

```

• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git branch -m main
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ echo "# Proyecto Git" > README.md

```

Genero un archivo de texto para utilizar durante la práctica , sumado a eso renombro la rama master a main para evitar problemas a futuro.

```

• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git status
En la rama main

No hay commits todavía

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
  README.md

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa "git add" para hacerles seguimiento)
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ 

```

Realizó un git status para ver el estado de los archivos en el repositorio.

Pregunta:

¿Qué información te brinda git status?¿Porque es útil antes de hacer un commit?

Git status te muestra el estado de los archivos, te muestra cuales archivos están sin guardar, cuales están guardados y cuales están listos para el commit. Puede utilizarse antes de commit para confirmar que archivos subirán al repositorio local.

Git add (Readme) - Git commit:

```

• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git add .
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git commit -m "Primer commit: README.md"
[main (commit-raiz) 6b58228] Primer commit: README.md
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git log
commit 6b58228bb0dd6535efd2a6c4cc038a015e8ff423 (HEAD -> main)
Author: elpeluca <agusnarsaniga@proton.me>
Date:   Sun May 17 18:32:18 2026 -0300

Primer commit: README.md

```

Añadir todos los archivos que no estén guardados en Git con el "." y los comiteo directamente al repositorio local.

Pregunta:

¿Cuál es la diferencia entre agregar (add) y guardar un cambio (commit)?¿Podés hacer un commit sin hacer un add antes?

Con git add preparamos qué archivos serán guardados en el repositorio de github, luego con commits los guardas en el repositorio local, puedes ejecutar el comando de commit sin hacer git add, pero solo te saltara un error con un mensaje de que no cargaste nada para commitear y te indicara el comando para hacerlo.

```

• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git add .
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
  modificados:   README.md

• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ 

```

Verificar con git status que archivos están preparados para commitear, al estar preparados los archivos aparecerán en verde.

Git status - Git log:

```
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git status
En la rama main

No hay commits todavía

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
  README.md

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa "git add" para hacerles seguimiento)
```

Realizó git para comprobar el estado de los archivos.

Pregunta:

¿Qué información muestra este comando?

El comando muestra que no tenemos archivos listos para commitear, lo que hace git status es mostrar el estado de los archivos dentro del repositorio y si han sufrido cambios.

```
commit 9e96046887a6bde472c0ff8d0ef45724de658a1b
Author: ElPelucaDLAP <agusnarsaniga@proton.me>
Date:   Mon May 18 02:52:14 2026 -0300

    Update README.md
```

Realizó git log para revisar log de commits pasados sobre esa rama.

Pregunta:

¿Qué información muestra este comando?

El comando muestra el primer commit donde se subió el README, también proporciona datos adicionales como sobre que rama se realizó el commit, en caso de no especificar.

Crea una carpeta vacía - utiliza **Git Status**:

```
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ mkdir test
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git status
En la rama main

No hay commits todavía

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
  README.md

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa "git add" para hacerles seguimiento)
```

Creo una carpeta vacía por medio de línea de comandos y luego por git status verificar si git la detecta, en este caso no la detecta, asumo que la carpeta no va a ser detectada ya que no contiene nada y git no la considero por eso.

Parte 2: Trabajo con ramas

Consigna 3: Crear y trabajar en una nueva rama

```
git branch tests
# (Crea una rama)
git checkout tests
#(Cambia a la rama)
```

```
git branch
```

```
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git branch tests
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git checkout tests
Cambiado a rama 'tests'
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git branch
main
* tests
```

Creó una rama llamada tests, me paso a la rama “tests” y listo las ramas donde me sale la que estoy parado.

```
git checkout -b desarrollo
# (Crea y cambia a la rama "desarrollo")
```

```
echo "Este archivo está en desarrollo." > desarrollo.txt
```

```
git add desarrollo.txt
```

```
git commit -m "Agrega archivo desarrollo.txt en rama desarrollo"
```

```
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git checkout -b "desarrollo"
Cambiado a nueva rama 'desarrollo'
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ echo "Este archivo está en desarrollo." > desarrollo.md
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git add desarrollo.md
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git commit -m "Agrega archivo desarrollo.md en rama desarrollo"
[desarrollo 664bd86] Agregar archivo desarrollo.md en rama desarrollo
1 file changed, 1 insertion(+)
create mode 100644 desarrollo.md
```

Género y me paso a una rama “desarrollo”, creó un archivo llamado desarrollo con un contenido, lo añadido a la cola de commit y lo comiteo.

```
git branch
```

```
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git branch
* desarrollo
main
tests
```

Listo las ramas que hay disponibles en el repositorio.

Pregunta:

¿Para qué sirve crear una nueva rama? ¿Qué ventajas tiene trabajar en ramas distintas en lugar de hacerlo siempre en la principal?

Una rama es un saliente de un código original, en él se puede hacer cambios y añadir nuevas funcionalidades, mientras que el código original no se ve afectado, lo que nos permite que los cambios esten aislados en un ambiente de pruebas propio o apartado y mantener una organización, esto ayuda a que los cambios puedan ser revisados después.

Consigna 4: Fusionar ramas

```
git checkout main
```

git merge desarrollo

(Fusione el contenido de la rama desarrollo a main)

```
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git checkout main
Cambiado a rama 'main'
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git merge desarrollo
Actualizando 6b58228..664bd86
Fast-forward
 desarrollo.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 desarrollo.md
```

Me muevo a la rama principal y fusiono los cambios de la rama desarrollo.

Pregunta:

¿Qué pasa si hay diferencias entre las ramas al hacer un merge? ¿Podés revertir un merge si algo sale mal?

Si a la hora de hacer merge entre las ramas una línea de la rama original fue modificada se nos pedirá intervenir y definir qué hacemos con los cambios, ya sea aceptar el nuevo, el viejo o fusionarlos y hacer uno nuevo. En caso de querer revertir un commit puedes utilizar git revert para revertir el último commit

lista las ramas

elimina la rama desarrollo

lista las ramas

```
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git branch
 desarrollo
* main
 tests
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git branch -d desarrollo
Eliminada la rama desarrollo (era 664bd86).
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git branch
* main
 tests
```

Listo las ramas que existen en mi repositorio, eliminó la rama desarrollo y vuelvo a listar las ramas para confirmar si desaparece.

Parte 3: Repositorio remoto y sincronización

git remote add origin https://github.com/usuario/practica-git

(Vincula tu repositorio local con el remoto)

(Debés reemplazar el usuario por tu nombre de usuario real de GitHub.)

git remote -v

```
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git remote add origin https://github.com/ElPelucaDLAP/practica-git
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git remote -v
origin https://github.com/ElPelucaDLAP/practica-git (fetch)
origin https://github.com/ElPelucaDLAP/practica-git (push)
```

Sincronizar el repositorio local con un repositorio remoto.

Pregunta:

¿Qué significa vincular tu repositorio local con uno remoto? ¿Podés trabajar sin conexión a internet después de hacer esto?

Vincular tu repositorio local con uno remoto es básicamente añadir otro gestor de versiones en la nube, tu gestor local se acomodará a él, podrás seguir trabajando sin conexión pero para sincronizar tu repositorio local con el remoto será necesario de esta.

crea una rama desde GitHub - desde la web. Con el nombre dev

`git pull`

`git branch`

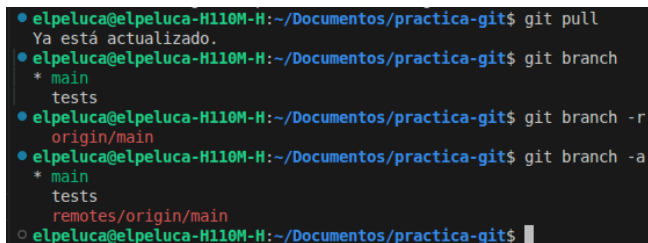
Listar ramas locales

`git branch -r`

Listar ramas remotas

`git branch -a`

Listar todas las ramas (locales y remotas)



```
elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git pull
Ya está actualizado.
elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git branch
* main
  tests
elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git branch -r
origin/main
elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git branch -a
* main
  tests
remotes/origin/main
elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$
```

Reviso que ramas locales tengo (branch), que ramas remotas tengo (branch -r) y que ramas en general tengo (branch -a).

Pregunta:

¿Qué sucede con las ramas remotas?

Las ramas remotas son originarias del servidor donde esté alojado el repositorio central, dichas ramas son solo de lectura para el repositorio local, lo que no significa que no podamos pushear cambios al servidor.

`git push -u origin master`

(Sube los commits locales a GitHub por primera vez)

Pregunta:

¿Qué estás logrando al hacer un push? ¿Por qué hay que subir los cambios al repositorio remoto si ya están en tu computadora?

Al realizar un push estamos enviando cambios al repositorio remoto, el subir los cambios al repositorio remoto permite poder trabajar en conjunto con otras personas y tener un backup en la nube del proyecto y todas sus versiones.


```

• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git add .
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git commit -m "cambio para subir commits a github"
[main 41de79f] cambio para subir commits a github
 1 file changed, 2 insertions(+), 4 deletions(-)
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git push -u origin master
error: src refspec master no concuerda con ninguno
error: falló el empuje de algunas referencias a 'https://github.com/ElPelucaDLAP/practica-git'
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git push -u origin main
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 375 bytes | 375.00 KiB/s, listo.
Total 3 (delta 0), reusados 0 (delta 0), pack-reusados 0
To https://github.com/ElPelucaDLAP/practica-git
   abf094f..41de79f  main -> main
rama 'main' configurada para rastrear 'origin/main'.

```

Subo los cambios a github, primero haciendo el commit local y luego pusheando a origen.

(En el sitio web de GitHub y modifica un archivo directamente desde el navegador (por ejemplo, README.md).

Volvé a tu terminal Git Bash)

git pull origin main

```

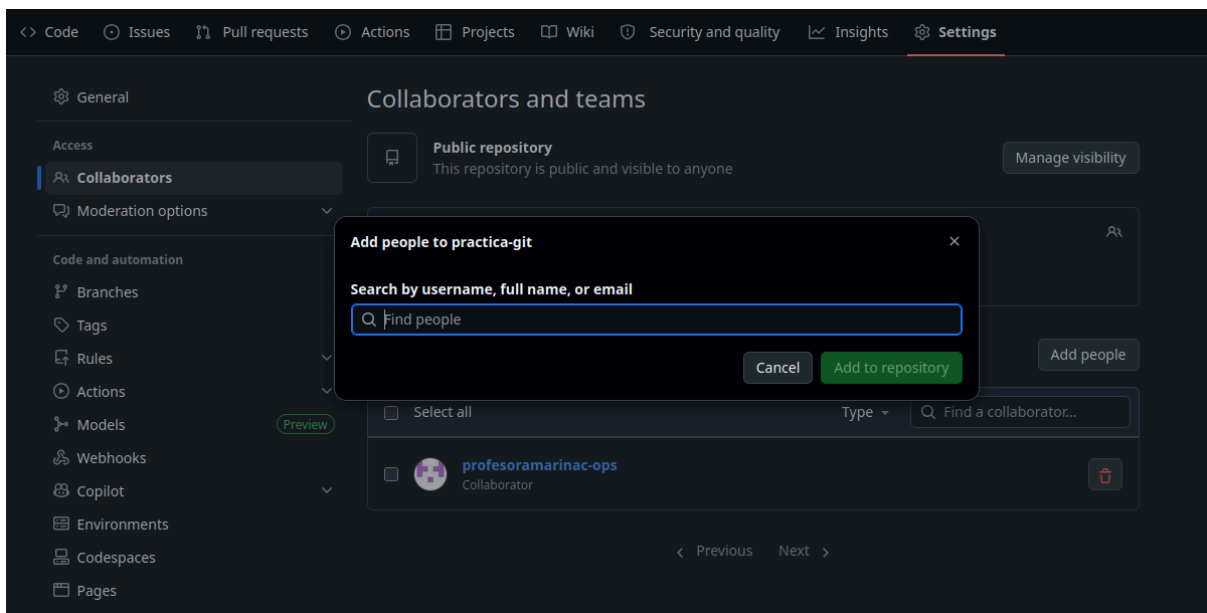
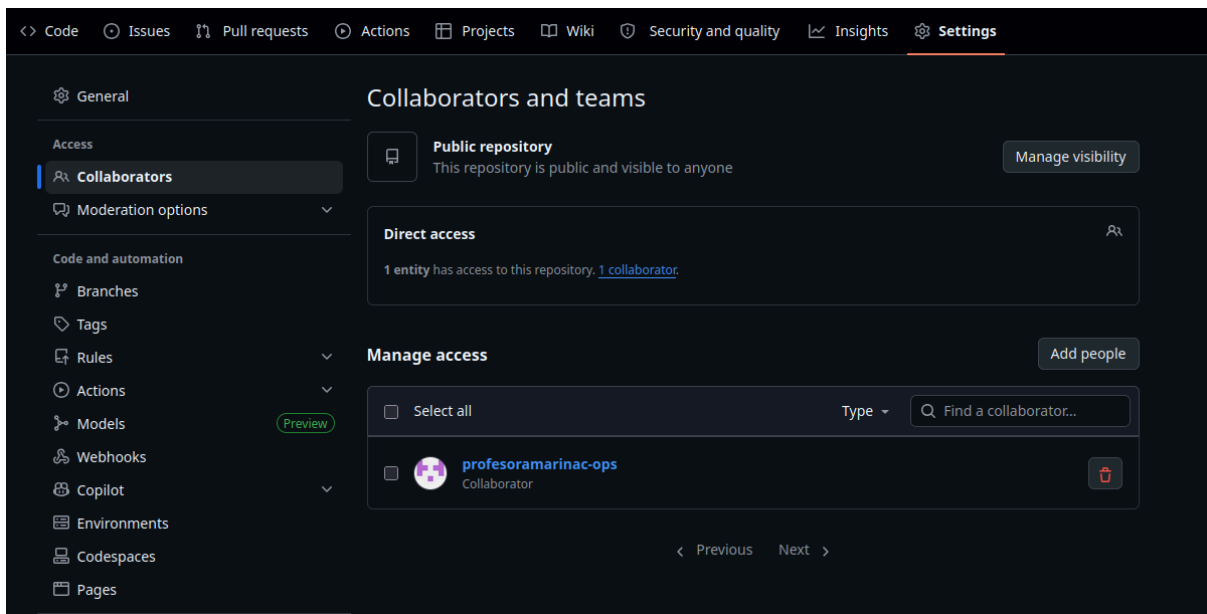
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git pull origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Desempaquetando objetos: 100% (3/3), 1015 bytes | 1015.00 KiB/s, listo.
Desde https://github.com/ElPelucaDLAP/practica-git
 * branch      main      -> FETCH HEAD
   664bd86..9e96046  main    -> origin/main
Actualizando 664bd86..9e96046
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)

```

Realizó cambios desde Github y los pulleo al repositorio local.

Parte 4: Colaboración y trabajo en equipo

Consigna 5: Agregar un colaborador



Consigna 6: Clonar un repositorio de un colaborador

`git clone https://github.com/[usuario]/practica-git.git`

```

• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web$ git clone https://github.com/profesoramarinac-ops/tpgithub2026
Clonando en 'tpgithub2026'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Recibiendo objetos: 100% (3/3), listo.
  
```

Clono repositorio de github por medio de una URL.

Pregunta:

¿En qué se diferencia clonar un repositorio de crear uno nuevo?

¿Por qué es importante clonar en lugar de descargar un ZIP?

Cuando clonas un repositorio recibís toda la información del repositorio, ya sea el historial el contenido y demás, cuando lo empiezas de 0 no tiene nada configurado y tú tendrás que irlo añadiendo

Cuando clonamos el repositorio en vez de descargar el zip estamos generando una conexión con el servidor que nos permitirá ir actualizando el contenido del repositorio a medida este se amplíe.

cd practica-git

```
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web$ cd tpgithub2026
```

git checkout -b correccion-companero

(Crea una rama nueva para hacer los cambios)

echo "Modificación realizada por el colaborador." >> README.md

git add README.md

git commit -m "Agrega modificación del colaborador"

git push origin correccion-companero

(Sube la nueva rama con cambios a GitHub)

```
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web$ cd tpgithub2026
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web/tpgithub2026$ git pull
Desde https://github.com/profesoramarinac-ops/tpgithub2026
* [nueva rama] correccion-compañero -> origin/correccion-compañero
Ya está actualizado.
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web/tpgithub2026$ git branch
* main
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web/tpgithub2026$ git branch -r
origin/HEAD -> origin/main
origin/correccion-compañero
origin/main
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web/tpgithub2026$ git checkout correccion-compañero
rama 'correccion-compañero' configurada para rastrear 'origin/correccion-compañero'.
Cambiado a nueva rama 'correccion-compañero'
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web/tpgithub2026$ echo "Modificación realizada por el colaborador." >> README.md
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web/tpgithub2026$ git add README.md
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web/tpgithub2026$ git commit -m "Realize cambios en el README"
[correccion-compañero f3834db] Realize cambios en el README
1 file changed, 1 insertion(+), 1 deletion(-)
• elpeluca@elpeluca-H110M-H:~/Escritorio/Desarrollo Web/tpgithub2026$ git push origin
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (2/2), listo.
Escribiendo objetos: 100% (3/3), 318 bytes | 318.00 KiB/s, listo.
Total 3 (delta 0), reusados 0 (delta 0), pack-reusados 0
To https://github.com/profesoramarinac-ops/tpgithub2026
30b3334...f3834db correccion-compañero -> correccion-compañero
```

(Otro Colaborador deberá ejecutar el siguiente comando)

git pull origin [rama]

(Trae y fusiona los cambios del remoto a tu rama local)

(Otro Colaborador deberá ejecutar el siguiente comando)

git fetch

(Descarga los cambios del remoto, sin fusionarlos)

Realize cambios en el README #1 

 Open EIPelucaDLAP wants to merge 1 commit into `main` from `correccion-compañero` 

 The head ref may contain hidden characters: `"correccion-compañero"`

Conversation 2 Commits 1 Checks 0 Files changed 1 +2 -1 

 EIPelucaDLAP commented 3 days ago Collaborator ...

Espero comprobaciones sobre los cambios hacia main

  Realize cambios en el README f3834db

  requested a review from **profesoramarinac-ops** 3 days ago

Pull request successfully merged and closed
You're all set — the branch has been merged.

 Add a comment

Write Preview H B I                               

Add your comment here...

 Markdown is supported  Paste, drop, or click to add files

 Close pull request Comment

Reviewers 

 **profesoramarinac-ops** 

Still in progress? [Convert to draft](#)

Assignees 

No one—[assign yourself](#)

Labels 

None yet

Projects 

None yet

Milestone 

No milestone

Development 

Successfully merging this pull request may close these issues.

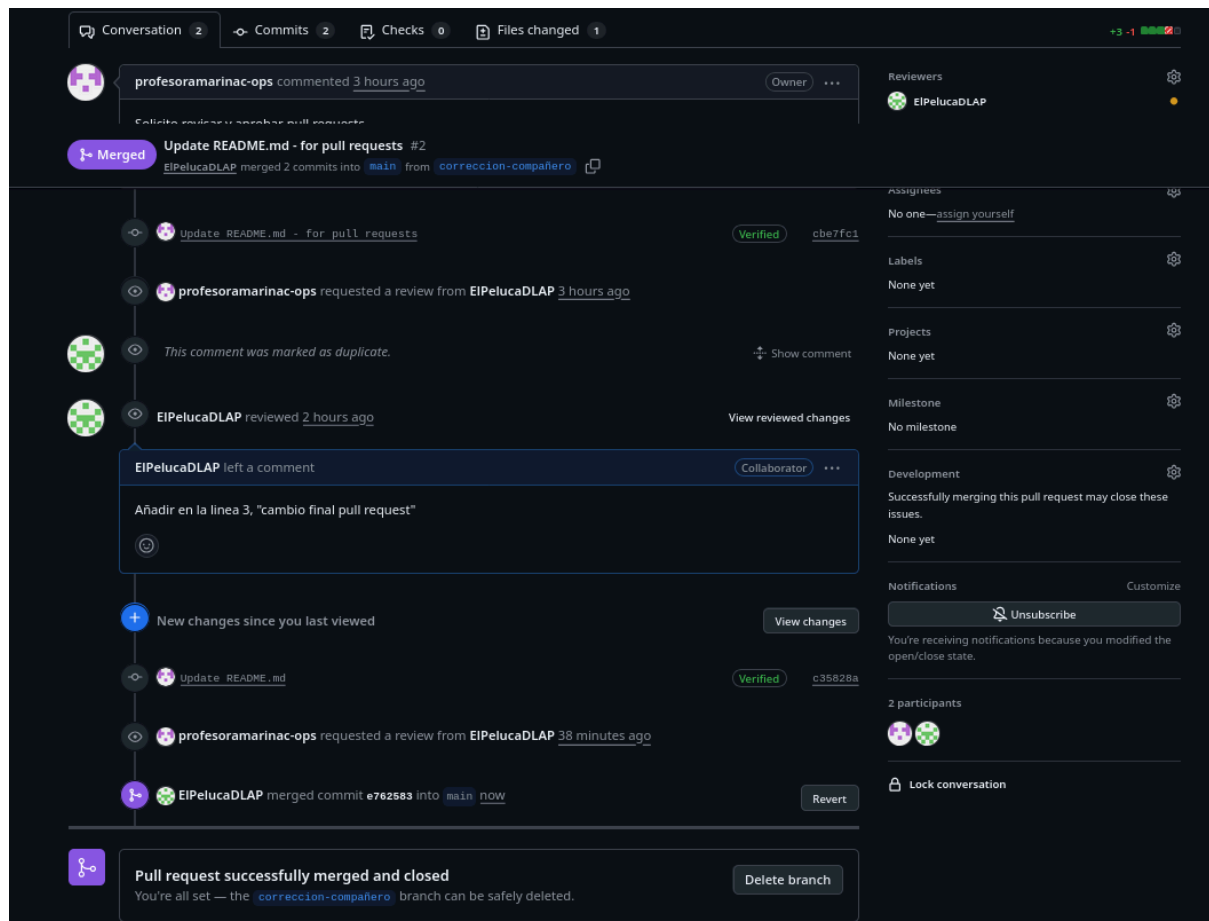
None yet

Notifications Customize

 Subscribe

You're not receiving notifications from this thread.

Captura de github, se hizo un pull request se pidieron cambio, se realizaron cambios y se volvieron a pushear.



Compañero realizó cambios y me seleccionó como revisor, pedí correcciones y posteriormente habilité el merge.

Pregunta:

¿Qué ventajas tiene hacer una revisión del código antes de fusionarlo? ¿Qué actitudes o prácticas colaborativas se ponen en juego al hacer un Pull Request?

Una revisión de código antes de la fusión por parte de un compañero permite prevenir errores o mejorar la calidad del código, así podemos aplicar Peer Code Review(revisión de código entre pares)y además de que permite que nuestro compañero adquiera conocimiento sobre el funcionamiento del código.

Parte 5: Conflictos y resolución

Consigna 7: Generando y resolviendo problemas de merge

Modifica la misma línea de un archivo en dos ramas diferentes para provocar un conflicto, luego resolverlo.

Paso a paso (Git Bash):

1. En la rama main:

git checkout main

echo "Línea original modificada en main" > conflicto.txt

git add conflicto.txt

git commit -m "Cambio en main que causará conflicto"

[Subirán los cambios al remoto]

```
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git checkout main
Ya en 'main'
Tu rama está actualizada con 'origin/main'.
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ echo "Línea original modificada en main" > conflicto.rd
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git add conflicto.rd
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git commit -m "Cambio en main que causará conflicto"
[main 6f3f5c5] Cambio en main que causará conflicto
1 file changed, 1 insertion(+)
create mode 100644 conflicto.rd
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git push origin
Enumerando objetos: 4, listo.
Contando objetos: 100% (4/4), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (2/2), listo.
Escribiendo objetos: 100% (3/3), 369 bytes | 369.00 KiB/s, listo.
Total 3 (delta 0), reusados 0 (delta 0), pack-reusados 0
To https://github.com/ElPelucaDLAP/practica-git
41de79f..6f3f5c5 main -> main
```

Genero cambios que puedan generar conflictos a la hora del merge.

2. En otra rama:

git checkout -b conflicto-rama

*(El parámetro -b significa "crear una nueva rama y cambiar a ella inmediatamente")

echo "Línea diferente modificada en rama" > conflicto.txt

git add conflicto.txt

git commit -m "Cambio en rama que causará conflicto"

[Subirán los cambios al remoto]

[Descargan los cambios al remoto]

git checkout main

git merge conflicto-rama

```

• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git checkout -b conflicto-rama
Cambiado a nueva rama 'conflicto-rama'
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ echo "Línea diferente modificada en rama" > conflicto.rd
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git add conflicto.rd
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git commit -m "Cambio en rama que causará conflicto"
[conflicto-rama ca7b08b] Cambio en rama que causará conflicto
1 file changed, 1 insertion(+), 1 deletion(-)

```

Género y me paro en una rama donde haré el conflicto, generó el conflicto y lo commiteo.

```

• elpeluca@Leno-Pelu:~/Escritorio/Gits/practica-git$ git merge conflicto-rama
Auto-fusionando conflicto.rd
CONFLICTO (contenido): Conflicto de fusión en conflicto.rd
Fusión automática falló; arregle los conflictos y luego realice un commit con el resultado.
• elpeluca@Leno-Pelu:~/Escritorio/Gits/practica-git$ git add conflicto.rd
• elpeluca@Leno-Pelu:~/Escritorio/Gits/practica-git$ git commit -m "resolviendo conflicto"
[main 5e5627e] resolviendo conflicto
• elpeluca@Leno-Pelu:~/Escritorio/Gits/practica-git$ git push origin
Enumerando objetos: 11, listo.
Contando objetos: 100% (11/11), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (9/9), listo.
Escribiendo objetos: 100% (9/9), 1.08 KiB | 370.00 KiB/s, listo.
Total 9 (delta 1), reusados 0 (delta 0), pack-reusados 0
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/ElPelucaDLAP/practica-git
52c4ddc..5e5627e main -> main
• elpeluca@Leno-Pelu:~/Escritorio/Gits/practica-git$

```

Se generará un conflicto. Abrir el archivo, resolverlo manualmente y luego:

git add conflicto.txt

git commit -m "Conflicto resuelto manualmente"

[Subirán los cambios al remoto]

Realizó solución manual del conflicto y hago un nuevo commit.

```

• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$ git push origin
Enumerando objetos: 13, listo.
Contando objetos: 100% (13/13), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (8/8), listo.
Escribiendo objetos: 100% (9/9), 987 bytes | 987.00 KiB/s, listo.
Total 9 (delta 0), reusados 0 (delta 0), pack-reusados 0
To https://github.com/ElPelucaDLAP/practica-git
76b009b..abf094f main -> main
• elpeluca@elpeluca-H110M-H:~/Documentos/practica-git$

```

Pusheo los cambios al repositorio remoto.

Pregunta:

¿Qué ocurre si otra persona hizo cambios en GitHub y vos no hiciste un pull antes de seguir trabajando? ¿Qué riesgos hay si no sincronizas antes de hacer cambios?

Antes de pushear tu código deberás pullear cambios externos y en caso de problemas subsanarlos de forma propia, en caso de no sincronizar antes de pushear git te bloqueará hasta que actualices tu repositorio local y resuelvas los problemas.

Reflexión:

Git es un sistema sumamente útil y, aun con todos los problemas que te puede llegar a generar al aprender a utilizarlo por línea de comandos, te permite adquirir costumbre sobre buenas prácticas a la hora de utilizarlo y conocer más comandos que los que podrías aprender utilizando la opción integrada en el IDE.

Link al repositorio:

<https://github.com/ElPelucaDLAP/practica-git>